AI Capstone HW1

112550169 潘仰祐

1. Implementation

Before the core code, there is an extra modification in `class Projection`'s `__init__`.

```
self.old_pixels = points # Added outside the TODO zone
```

I add this line to provide sufficient information for the core code to work.

Core code

My core code is in the predefined top_to_front() function from the template. It involves three steps:

Step 1: revert the points on top-view to cam2's coordinates

```
def top_to_front(self, theta=-90, phi=0, gamma=0, dx=0, dy=1.5, dz=0, fov=90):
    """...

    # Step 1: revert the points on top-view to cam2's coordinates
    old_pixels = np.array(self.old_pixels)

    # - from [0~511, 0~511] to [-2.5~2.5, -2.5~2.5]
    old_point_offsets = old_pixels / 512.0 * 5.0 - 2.5
    old_point_offsets[:, 1] *= -1

    # - insert z-coordinate such that (x, y) -> (x, y, -2.5)
    points_relToCam2 = np.insert(old_point_offsets, 2, -2.5, axis=1)
```

Here, it takes the clicks of pixels, assuming the floor is 2.5 units away from the camera, and reconstruct the 3D local coordinates relative to the Camera 2 (the top-view/BEV camera).

Step 2: rotate the points to cam1's coordinates

```
# Step 2: rotate the points to cam1's coordinates

# - apply reverse rotation from pitch(theta), yaw(phi), and roll(gamma)
rad_x = np.deg2rad(theta)
rad_y = np.deg2rad(phi)
rad_z = np.deg2rad(gamma)

R_x = np.array([ ...

R_y = np.array([ ...

R_z = np.array([ ...

R = R_x @ R_y @ R_z

# - get the points relative to camera 1 by applying translation AFTER rotation
points_rotated = points_relToCam2 @ R.T
points_relToCam1 = points_rotated + np.array([dx, dy, dz])

print(points_relToCam2)
print(points_rotated)
print(points_relToCam1)
```

Then, instead of using the world coordinates, I tried to do a two-stages transformation to directly convert the points to the local coordinate relative to Camera 1 (the front camera).

- The first stage is to do rotation around Camera 2, so the temporary local coordinate is at Camera 2 but oriented the same way as Camera 1.
- The second stage is to add a simple offset to move the origin from Camera 2 to Camera 1.

For easily debugging, I printed the points for each stage out. I would like to discuss the order of these two stages later.

Step 3: mimicking the pin-hole camera with matrix to project the points to front-view

```
# Step 3: mimicking the pin-hole camera with matrix to project the points to front-view

# - check if any points are behind or exactly on the camera plane
#   (Z >= 0, since camera faces -Z)
if np.any(points_relToCam1[:, 2] >= 0):
    print("\n[WARNING] Some selected points are behind Camera 1! Projection will be distorted.")

# - mimicking by "divide by -z" and "multiply by f"
#   NOTE: spec says we have 512x512 output
f = (512 / 2) / np.tan(np.deg2rad(fov / 2))
cx, cy = 256.0, 256.0 # 256 = 512 / 2

u = f * (points_relToCam1[:, 0] / -points_relToCam1[:, 2]) + cx
v = f * (-points_relToCam1[:, 1] / -points_relToCam1[:, 2]) + cy

new_pixels = np.column_stack((u, v)).astype(int).tolist()

print(new_pixels)

return new_pixels
```

In the third step, I did the step 1 reversely. It divides the `(x, y)` of each point by their own `-z` to simulate the perspective view. (Here we use -z since the Camera is looking towards -z, so we need "-" to cancel it out).

Then, it multiplies the new "(X, Y) in the ratio of distance" coordinates by f and add an offset (256, 256) to ensure that things in a ratio range can be displayed in the 512x512 canvas. In this case that fov = 90deg, f = 256 and the displayed ratio range is (-1.0~1.0, -1.0~1.0). For simplicity, I split the columns of x and y to manipulate them separately.

Finally, the output is combined and stored in the numpy matrix `new_pixels` and returned.

Result and Discussion

The result

The order of rotation and translate stages

During the implementation phase, I was kind of dizzy by maths and accidentally did the translation from Camera 2's origin to Camera 1's before rotation. This is wrong because, if we do the rotation after translation, the whole coordinates are pushed away and will rotates around Camera 1.

Consider Camera 1, Camera 2 and a temporary camera in the following graphs, where the right is the camera's orientation, the `X` is the camera, and `V` and `>` are pointing to the ground.

- If we do rotation before translation:

```
<Camera2>     <CamTemp>   <Camera1>

 X  >     |     X     |
          |           |     X
          |     V     |     V
```

- But if we do it in the wrong way that translation is before rotation

```
<Camera2>     <CamTemp>   <Camera1>

 X  >     |       >   |
          |     X     |     X
          |           |       V
```

This clearly shows the wrong order cause an issue.

The z's issue

When some of the selected points are "after" the camera plane (z = 0), the rendering will be wrong. For example:

The Camera Plane ⟶ ⇒

That's an awkward situation. A standard solution should do clipping before the projection to canvas, but this homework doesn't require it according to the Q&A Page. To hint the user that happens, I added a warning message output, for example:

```
Note that Qt no longer ships fonts. Deploy some (from https://dejavu-fonts.github.io/ for example) or switch to fontconfig.
289   170
146   339
328   349
493   171
[[ 0.32226562  0.83984375 -2.5       ]
 [-1.07421875 -0.81054688 -2.5       ]
 [ 0.703125   -0.90820312 -2.5       ]
 [ 2.31445312  0.83007812 -2.5       ]]
[[ 0.32226562 -2.5        -0.83984375]
 [-1.07421875 -2.5         0.81054687]
 [ 0.703125   -2.5         0.90820312]
 [ 2.31445312 -2.5        -0.83007813]]
[[ 0.32226562 -1.         -0.83984375]
 [-1.07421875 -1.          0.81054687]
 [ 0.703125   -1.          0.90820312]
 [ 2.31445312 -1.         -0.83007813]]

[WARNING] Some selected points are behind Camera 1! Projection will be distorted.
[[354, 560], [595, -59], [57, -25], [969, 564]]
```

References

- Introduction to Computer Graphics' Slides

2. Questions

a. How do the intrinsic and extrinsic matrices contribute to the BEV-to-front-view projection?

An extrinsic matrix converts (transforms) the points between two 3D coordinate systems, while an intrinsic matrix projects the 3D space to a 2D plane according to the camera parameters it wants to emulate.

b. What factors may cause projection misalignment? List at least two factors and explain the reason.

I came up with three potential reasons:

- There may be some rounding errors during the transformations (Which should be small and not so noticeable to humans).
- Since these images are synthesized from real cameras, and physical things have their own deviations, there may be some offset during the camera focusing and indoor localization. For example, ideally they should define the camera's projection to the floor to be y = 0, but it may actually be y = 0.1 or -0.1, which is small but still perceptable.
- The ground may be visibly flat but not really flat, or there is a slope. This may cause a piece of the ground looks farther and bigger (since it

has a +y drift to the camera's canvas), making the selected and transformed zone looks nearer and smaller.